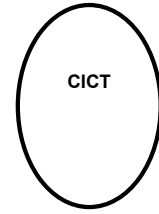


West Visayas State University
COLLEGE OF INFORMATION AND COMMUNICATIONS TECHNOLOGY

Luna St., La Paz, Iloilo City 5000 Iloilo, Philippines
Telephone: (033) 320-0870 Website: www.wvsu.edu.ph



**TulongText PH: Filipino-English Disaster Post
Classification and Urgency Triage System**

CCS 229 - Natural Language Processing

Final Project

Submitted by:

Baladjay, Aser Jr. S.

Dorado, Louise Marielle V.

Montenegro, Karlo Roel

Pontillas, Steven Ken E.

BSCS 3A AI

Submitted to:

[Instructor Name]

CCS 229 - Instructor

May 22, 2026

Project Overview

Introduction to the Project

TulongText PH is a Filipino-English disaster relief post classification and urgency triage system developed as a Natural Language Processing final project. The system focuses on disaster-related social media posts, especially posts written in English, Filipino, or Taglish. During emergencies such as typhoons, floods, earthquakes, and other crisis events, social media becomes one of the fastest channels for public updates. However, these posts are often noisy, informal, incomplete, multilingual, and mixed with irrelevant or promotional content.

The purpose of TulongText PH is to help organize these posts into meaningful humanitarian categories. Instead of manually reading and sorting each post, the system uses NLP classification models to identify whether a post is related to rescue needs, medical concerns, evacuation, infrastructure damage, warnings, donations, general updates, or non-humanitarian content. It also assigns an urgency level based on the predicted category, allowing the output to be more useful for triage and prioritization.

The project includes the complete NLP workflow: dataset collection, dataset normalization, text preprocessing, label mapping, model training, model comparison, evaluation, backend API development, and web application implementation. The final application allows a user to enter a disaster-related post and receive a predicted category, urgency level, confidence score, preprocessing details, and top predictions through a simple browser interface.

Problem Statement

Disaster-related social media posts are difficult to process manually because they are often written in different languages, formats, and levels of urgency. A single emergency event can generate hundreds or thousands of posts. Some posts may contain urgent rescue requests, while others may be general updates, warnings, donation offers, or irrelevant advertisements. For local government units, response teams, volunteers, and monitoring teams, manually sorting these posts at scale can be slow and overwhelming.

This creates a problem in disaster-response communication. Important posts may be buried among unrelated or less urgent information. A post asking for rescue, reporting injuries, or identifying damaged infrastructure may need faster attention than a general reaction or promotional post. Without an automated way to organize text posts, response teams may spend

valuable time filtering information instead of acting on it.

TulongText PH addresses this problem by using NLP to automatically classify disaster-related posts into humanitarian categories and urgency levels. The system is not intended to replace human responders. Instead, it acts as a triage assistant that helps surface potentially important posts more quickly and supports faster information organization during disaster situations.

Significance in Natural Language Processing

This project is significant in Natural Language Processing because it demonstrates how text classification can be applied to a real-world humanitarian problem. Social media posts are unstructured text data. They may include informal words, misspellings, code-switching, local expressions, hashtags, locations, and incomplete sentences. NLP techniques are needed to clean, normalize, tokenize, and transform these posts into features that machine learning models can understand.

The project also highlights the challenge of multilingual and Taglish disaster communication. Unlike clean formal documents, disaster posts often contain mixed English and Filipino phrases such as "Need rescue sa barangay," "baha na hanggang bubong," or "evacuation center open na." The system must handle this noisy language while still identifying the humanitarian intent of the message.

Dataset Sources and Usage

The project uses multiple disaster and humanitarian-response datasets. The stable dataset had 220,495 original category rows and 164,075 rows after quality filtering and deduplication.

Dataset / Source	Role in Project	Rows / Usage	Notes
CrisisBench-all-lang	Main humanitarian categories	97,929 rows in clean subset	Large source for crisis labels
HumAID-events	Humanitarian event categories	61,203 rows in clean subset	Core category training source
CrisisLexT26	Disaster tweet data	4,943 rows in clean subset	Crisis-event social posts
SEACrowd Yolanda	Local relevance support	735 sentiment rows	Documented, not main category training
Disaster Response Messages	Expansion experiment	5,469 unique rows	Imported but not final model source
CrisisMMD	Expansion experiment	38 unique rows	Mostly overlaps with CrisisBench

Final Label and Urgency Mapping

The final taxonomy was simplified into eight humanitarian categories. Urgency is derived from the predicted category to make the triage logic transparent.

Final Category	Urgency	Description
rescue_or_urgent_needs	Critical	Rescue requests, urgent needs, stranded people
medical_or_casualties	Critical	Injuries, deaths, medical support, missing people
evacuation_or_displacement	High	Evacuation centers, shelters, displaced residents
infrastructure_damage	High	Roads, bridges, utilities, landslides, damaged buildings
warnings_or_advice	Moderate	Alerts, advisories, caution and safety instructions
donation_or_volunteering	Moderate	Relief goods, donations, volunteers, supplies
general_update	Low	Relevant but non-urgent updates or reactions
not_humanitarian	Low	Unrelated, promotional, or non-triage content

NLP Task Selection

The main NLP task selected for this project is multi-class text classification. Each input post is classified into one of eight final humanitarian categories: `rescue_or_urgent_needs`, `medical_or_casualties`, `evacuation_or_displacement`, `infrastructure_damage`, `warnings_or_advice`, `donation_or_volunteering`, `general_update`, and `not_humanitarian`.

A secondary task is binary actionability classification. This model predicts whether a post is actionable or non-actionable for disaster triage. Actionable posts include rescue, medical, evacuation, infrastructure, warning, and donation-related content. Non-actionable posts include general updates and posts that are not humanitarian.

Urgency classification is implemented as a rule-based layer derived from the predicted category. Rescue and medical categories are considered critical. Evacuation and infrastructure categories are considered high urgency. Warnings and donations are considered moderate urgency. General updates and non-humanitarian posts are considered low urgency.

Dataset Cleaning and Label Mapping

A major part of the project involved cleaning and normalizing labels from multiple datasets. Since different datasets use different category names, the labels were mapped into a simplified final taxonomy suitable for the web app and presentation. This mapping was necessary to create a consistent classification problem.

Several ambiguous source labels were handled carefully. The `affected_individuals` label was not automatically treated as medical unless injury or death was explicit. The `response_efforts` label was mapped to `donation_or_volunteering` when response or aid terms were present, and otherwise mapped to `general_update`. The `rescue_volunteering_or_donation_effort` label was split using keyword logic to distinguish rescue-related content from donation or volunteer-related content. The `terrorism_related` label was dropped from the clean subset because it was too broad for the project's disaster-response triage categories.

Preprocessing Pipeline

The preprocessing pipeline prepares raw social media text for model training and prediction. It includes text cleanup, normalization, tokenization, and feature preparation. URLs, mentions, repeated whitespace, and unnecessary noise are cleaned. The text is lowercased and transformed into a more consistent representation. Hashtags and disaster-related keywords are

preserved where useful because they may contain important category information.

For display in the web app, the system shows the original text, cleaned text, and token information. For the baseline model, the cleaned text is converted into TF-IDF features. The final selected baseline uses word bigram features, which allow it to capture short but meaningful phrases such as "need rescue," "evacuation center," "power lines," and "relief goods."

Model Development

Two main model types were developed and compared: a traditional machine learning baseline and a transformer model. The baseline model uses TF-IDF feature extraction with a calibrated LinearSVC classifier. Several baseline candidates were tested, including logistic regression, LinearSVC, SGD classifier, Complement Naive Bayes, word n-grams, and word-character feature variants.

The transformer model uses a locally fine-tuned multilingual DistilBERT sequence classifier. This was included to compare a deep learning model against the traditional baseline. The transformer does not use GPT or any external prediction API. It is a pretrained language model with a locally trained classification head.

A secondary actionability model was also developed using classical text classification. This model predicts whether a post is actionable or non-actionable. It supports the main triage goal by providing an additional binary signal that may be easier to interpret in emergency monitoring.

Feature and Model Comparison

This table compares how each model represents text and why each model was included in the project.

Model	Text Features / Representation	Classifier	Strengths	Limitations	Final Role
Baseline	TF-IDF word bigrams	Calibrated LinearSVC	Fast, stable, best saved metrics	Less contextual understanding	Final deployed model
Transformer	Multilingual token embeddings	DistilBERT sequence classifier	Can model context and multilingual patterns	Slow local training, lower saved result	Comparison model
Actionability	Word + character TF-IDF	SGD classifier	Strong binary triage result	Only actionable vs non-actionable	Secondary output

Model Evaluation Summary

The final baseline model was selected because it achieved the strongest saved result for the eight-class classification task while remaining fast and stable for a live demo.

Model	Accuracy	Macro Precision	Macro Recall	Macro F1	Weighted F1
Baseline TF-IDF + Calibrated LinearSVC	0.7600	0.7654	0.7733	0.7689	0.7576
Transformer DistilBERT Multilingual	0.6661	0.6600	0.6661	0.6608	N/A
Actionability Classifier	0.8683	0.8535	0.8690	0.8596	0.8697

Class Distribution in Clean Training Subset

The clean subset remains imbalanced, which is one reason macro-averaged metrics are important. Macro metrics treat all classes more equally than accuracy alone.

Final Label	Rows in Clean Subset
general_update	53,559
not_humanitarian	52,225
rescue_or_urgent_needs	14,289
donation_or_volunteering	12,607
infrastructure_damage	11,295
medical_or_casualties	9,584
warnings_or_advice	6,666
evacuation_or_displacement	3,850

Model Comparison

The model comparison showed that the TF-IDF baseline outperformed the saved transformer on the current evaluation results. Although transformer models are often stronger for language understanding, performance depends on dataset quality, training time, compute resources, and label consistency. In this project, the baseline model performed better because the disaster categories are strongly associated with keyword and phrase patterns.

The transformer model may improve with more stable training, stronger compute resources, checkpoint saving, larger training runs, and more carefully reviewed labels. However, for the final demonstration, the baseline model is the best choice because it is faster, more stable, easier to deploy, and has the highest saved evaluation score.

System Architecture

The system follows a simple end-to-end architecture. First, datasets are imported and normalized into processed CSV files. Second, preprocessing scripts clean the text and prepare labels. Third, training scripts build and evaluate the baseline, transformer, and actionability models. Fourth, saved model artifacts are loaded by the Flask backend. Finally, the browser frontend sends user input to the backend and displays the prediction results.

The backend is responsible for loading models, preprocessing text, running predictions, applying keyword support, deriving urgency, and returning structured JSON responses. The frontend is responsible for user interaction, demo examples, model selection, result display, confidence bars, category cards, and the visual dashboard.

Web App Implementation

The web application was implemented using plain HTML, CSS, and JavaScript for the frontend, and Python with Flask and Flask-CORS for the backend. The frontend provides a polished TulongText PH dashboard where users can type or paste a disaster-related post, select a model, and click Analyze.

The application displays the predicted humanitarian category, urgency badge, confidence score, cleaned text, token count, top predictions, and actionability result. It also includes clickable demo examples for rescue, medical, evacuation, infrastructure damage, and non-humanitarian posts.

The Flask backend exposes four main API endpoints: GET /api/health, GET /api/models, POST

/api/predict, and POST /api/batch_predict. The backend also serves the frontend directly, allowing users to open the app through the Flask URL instead of manually opening the HTML file.

Functional Requirements

The system must allow users to enter a disaster-related social media post and receive a predicted category and urgency level. It must support model selection between available category models. It must display confidence scores, top predictions, cleaned text, token count, actionability, and inference time. It must also handle empty input with a friendly validation message.

The web app must connect to the Flask backend and preserve the API endpoints required for prediction. Demo examples must populate the input field and return the user to the analysis section. The system should support English, Filipino, and Taglish examples as much as the available training data allows.

Hardware Requirements

Processor: A modern dual-core CPU is sufficient for running the baseline model and web application. The baseline model is lightweight and can perform predictions quickly on CPU.

RAM: At least 8 GB RAM is recommended for running the Flask backend, loading model artifacts, and performing local testing. More memory may be useful when training on large processed datasets.

Disk Space: At least 5 to 10 GB of available disk space is recommended for datasets, processed CSV files, model artifacts, evaluation outputs, and documentation.

Special Considerations: A GPU is not required for running the final baseline model. A GPU is helpful for transformer fine-tuning, but the final deployed model can run on CPU.

Software Requirements

Operating System: The project was developed and tested on Windows using a local Python environment.

Development Environment: Visual Studio Code was used for editing and running the project files. Git and GitHub can be used for version control and submission.

Programming Language: Python is used for dataset processing, preprocessing, model training, evaluation, and backend development. HTML, CSS, and JavaScript are used for the frontend web interface.

Main Libraries: Flask, Flask-CORS, scikit-learn, pandas, numpy, joblib, torch, transformers, matplotlib, and related dependencies listed in requirements.txt.

Testing and Validation

Testing was performed across dataset, model, API, and frontend components. Dataset checks confirmed that processed files could load successfully, required text and label fields were present, and normalized duplicates were reduced in the clean subset. Class distributions were generated to understand imbalance.

Model evaluation included accuracy, macro precision, macro recall, macro F1-score, weighted F1-score, classification reports, confusion matrices, and model comparison tables. The baseline, transformer, and actionability models were evaluated separately.

API checks verified that `/api/health`, `/api/models`, `/api/predict`, and `/api/batch_predict` work as expected. Frontend checks verified that the web interface contains the required elements, calls the backend, displays prediction results, and handles demo examples correctly.

Error Analysis

The most difficult part of the task is the overlap between humanitarian categories. For example, a post asking for food, water, and rescue may contain both urgent-needs language and donation-related language. Warnings and general updates can also overlap when a post simply informs people about danger without directly asking for help.

The `general_update` category is broad, which makes it difficult to distinguish from `warnings_or_advice` or `not_humanitarian` in some cases. Taglish and local Filipino expressions also remain challenging because the amount of directly labeled local data is limited.

These issues explain why an honest 90 percent score is difficult for the eight-class task. The project addresses this by reporting realistic metrics, improving data quality, and adding a secondary actionability model for broader triage.

Current Limitations

The urgency level is currently rule-based rather than independently predicted by a separate urgency model. This makes the logic transparent but may miss cases where the wording of a post suggests higher or lower urgency than the category alone.

Location extraction is not yet implemented. The system does not automatically identify barangays, cities, provinces, coordinates, or evacuation centers as structured location fields. It also does not verify whether a post is true or current.

The transformer model was limited by local compute constraints. A longer training run was interrupted before completion, and the saved transformer did not outperform the baseline. Filipino and Taglish labeled data are also still limited compared with English crisis data.

Future Improvements

Future improvements include adding location extraction for barangays, cities, and provinces; creating a batch triage dashboard; adding CSV upload and export; improving explainability by highlighting influential words; adding confidence calibration and low-confidence warnings; and integrating near-duplicate detection.

Further model improvements may include training XLM-RoBERTa or another stronger multilingual transformer, adding checkpoint saving to prevent lost training progress, using a manually reviewed validation set, and collecting more Filipino and Taglish disaster posts with humanitarian category labels.

Conclusion

TulongText PH demonstrates how Natural Language Processing can be applied to disaster-response communication. The project converts noisy social media text into structured humanitarian categories and urgency levels. It includes dataset preparation, preprocessing, label mapping, model comparison, evaluation, API development, and a working web application.

The final selected model is the TF-IDF word bigram baseline with calibrated LinearSVC because it achieved the strongest saved performance and is stable for a live demo. The transformer and dataset expansion experiments were still valuable because they showed the importance of evaluation, compute constraints, and data quality.

References

CrisisLexT26 disaster tweet dataset. HumAID-events dataset from QCRI. CrisisBench-all-lang dataset from QCRI. SEACrowd Typhoon Yolanda tweets dataset. HFAbrar Disaster Response Messages dataset. QCRI CrisisMMD dataset. Scikit-learn documentation for TF-IDF, LinearSVC, SGDClassifier, ComplementNB, and calibration. Hugging Face Transformers documentation for multilingual DistilBERT sequence classification. Flask and Flask-CORS documentation for backend API development.